

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
∞·····☼·····∞



BÁO CÁO
MILESTONE 2
LỚP L02--- NHÓM 11 --- HK251

Sinh viên thực hiện	Mã số sinh viên	Điểm số
TRƯƠNG THÀNH NHÂN	2312454	
TRƯƠNG TUÂN ĐẠT	2310715	
ĐINH TRẦN QUỐC AN	2310005	

Thành phố Hồ Chí Minh – 2025

MỤC LỤC

1. Introduction.	2
2. Design Strategy.	2
2.1. ALU and Branch Comparison.	2
❖ Phân tích yêu cầu thiết kế alu:	2
❖ Phân tích yêu cầu thiết kế Branch Comparison:	4
2.2. Regfile and Load-Store Unit.	6
❖ Phân tích yêu cầu thiết kế Regfile:	6
❖ Phân tích thiết kế Instruction memory và Data memory:	7
2.3. Single-Cycle Processor.	9
❖ Phân tích yêu cầu thiết kế Single-Cycle Processor:	9
❖ Sơ đồ khối của Single-Cycle Processor:	9
3. Application	10
3.1 LED 7 đoạn : swith nhị phân ra led 7 đoạn thập phân	10
3.2 Led red : tính toán 4 bit + -	17

1. Introduction.

Báo cáo thí nghiệm này trình bày về thiết kế của một bộ xử lý single cycle RV32I và có thể được tổng hợp trên FPGA. Báo cáo mô tả về baseline functionality, verification quality, application demonstration, và alternative designs của bộ xử lý. Baseline functionality cho thấy cách bộ xử lý đáp ứng các yêu cầu của RV32I và cách nó có thể được triển khai trên tài nguyên của FPGA. Verification quality cho thấy cách bộ xử lý được kiểm tra bằng cách sử dụng một bộ kiểm tra toàn diện bao gồm một số trường hợp đủ và sử dụng một chiến lược xác minh hợp lý. Application demonstration cho thấy cách bộ xử lý có thể chạy một ứng dụng đơn giản thực hiện các phép toán số và truy cập bộ nhớ. Các thiết kế thay thế cho thấy cách bộ xử lý có thể được cải thiện bằng cách thêm các tính năng đặc biệt.

2. Design Strategy.

2.1. ALU and Branch Comparison.

❖ *Phân tích yêu cầu thiết kế alu:*

Bộ ALU (Arithmetic Logic Unit) trong một CPU RV32I thực hiện các phép toán cơ bản như cộng, trừ, AND, OR, và các phép toán logic bitwise dựa trên tín hiệu điều khiển. Bộ ALU nhận dữ liệu đầu vào từ thanh ghi, thực hiện phép toán theo tín hiệu điều khiển và xuất kết quả.

Và đây là các phép tính mà alu cần đáp ứng:

ADD/SUB : dùng full_adder để làm bộ cộng trừ

- Cộng khi Cin =0
- Trừ khi Cin=1 đổi B thành B'

SLR/SLL : dùng 32 case để dịch thủ công

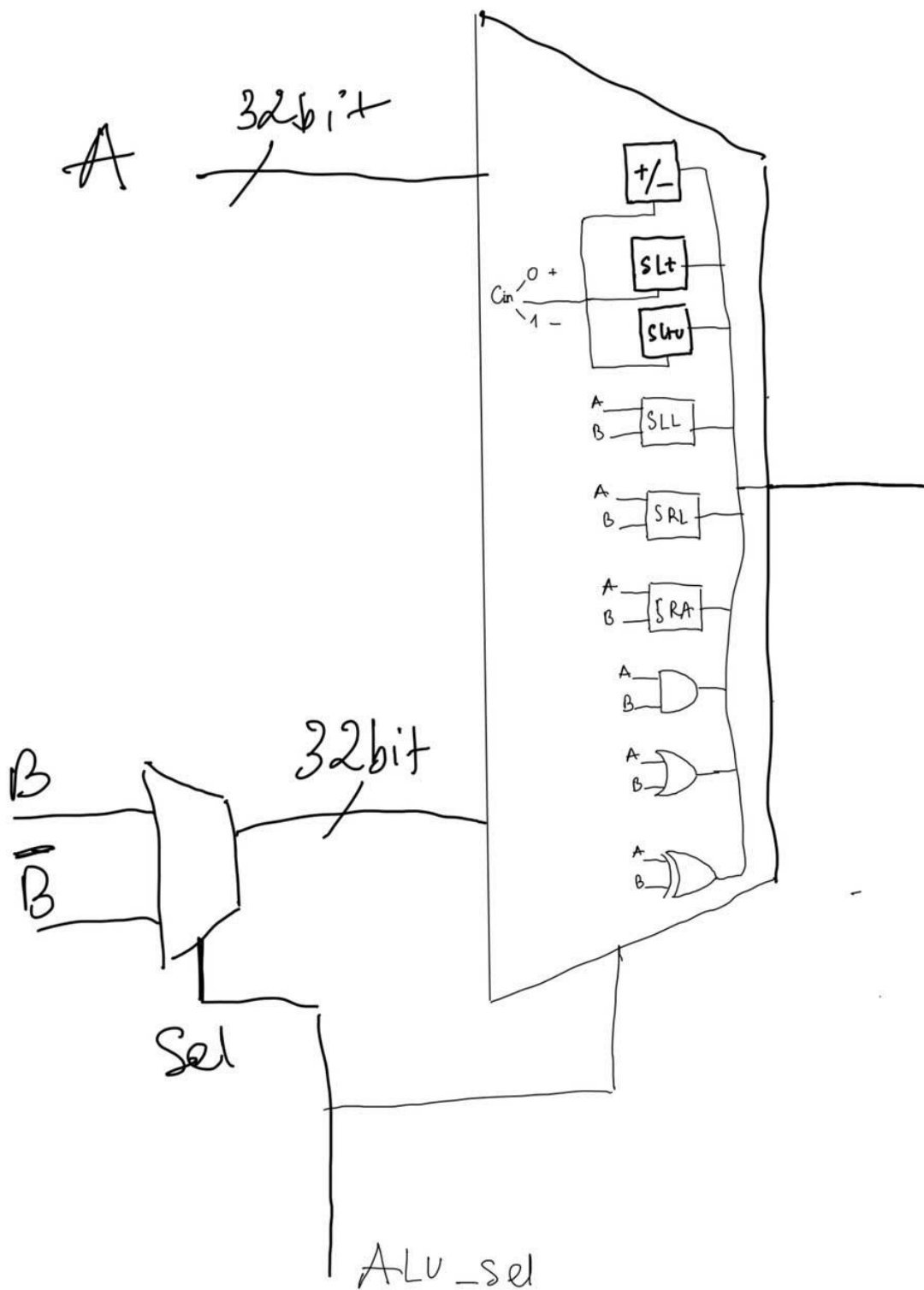
SRA : tương tự như SLR nhưng mở rộng bit MSB khi dịch

AND /OR : dùng phép toán tử cơ bản như : | &

XOR: là trừ bộ xor 1 bit → 32 bit

SLT : phép Xor 2 biến kế cuối và kết quả

SLTU : lấy bù cò carry top đầu



❖ *Phân tích yêu cầu thiết kế Branch Comparison:*

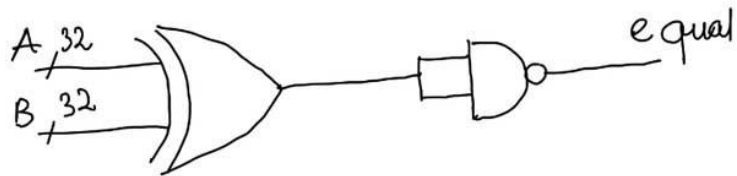
Branch Comparison được thiết kế để thực hiện so sánh giữa hai toán hạng `rs1_data` và `rs2_data`. Có hai đầu ra: `br_less` để xác định nếu $A < B$ và `br_equal` để xác định nếu $A = B$. Nếu tín hiệu `br_unsigned` là 1, so sánh sẽ được thực hiện theo kiểu không dấu (unsigned comparison). Ngược lại, nếu `br_unsigned` không phải là 1, so sánh sẽ được thực hiện theo kiểu có dấu (signed comparison).

Khối này sẽ xác định kết quả của so sánh và cập nhật `br_less` và `br_equal` dựa trên kết quả của việc so sánh A và B theo các quy tắc của so sánh không dấu hoặc có dấu tùy thuộc vào giá trị của `br_unsigned`. Cũng như khi thiết kế bộ ALU, chỉ được sử dụng phép toán cộng trong SystemVerilog operations (không được sử dụng các phép toán `-`, `>` và `<`). Nên muốn sử dụng phép toán $a - b$, ta chỉ cần lấy bù 2 của b, cụ thể như sau $a - b = a + b + 1$.

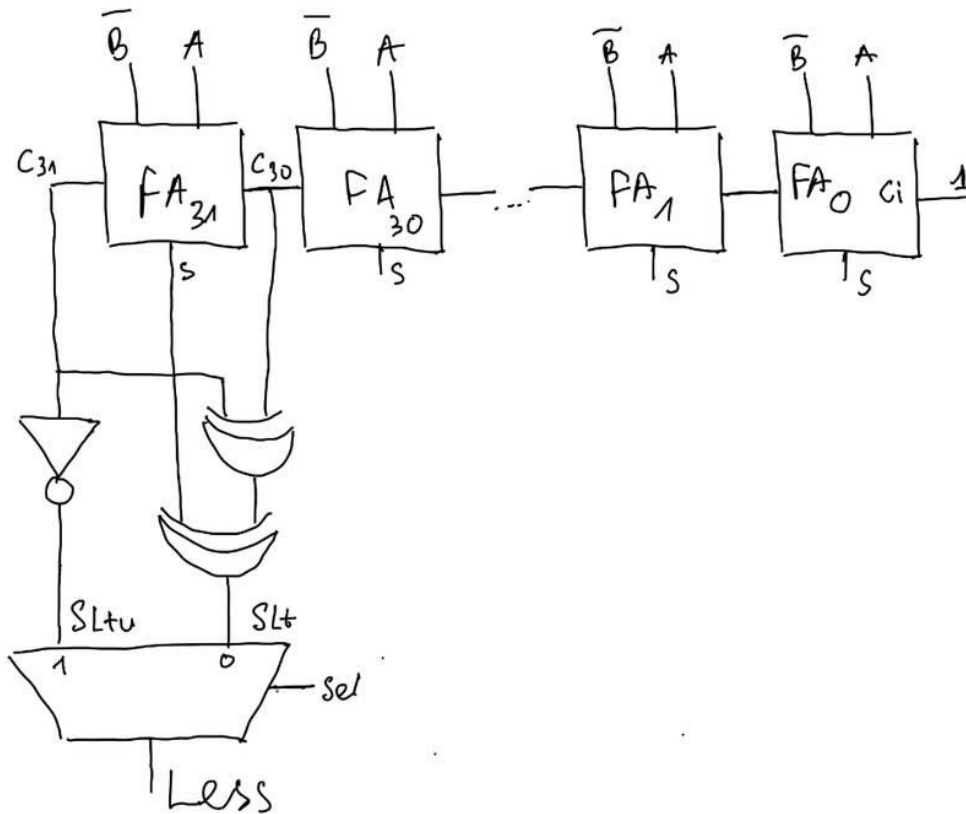
Equal : xor từng bit nếu = 0 ➔ đảo kết quả lên 1

Less: sử dụng lại bộ trừ full_adder 32 bit và dùng lại cờ carry và overload để xác định unsigned và signed

Equal



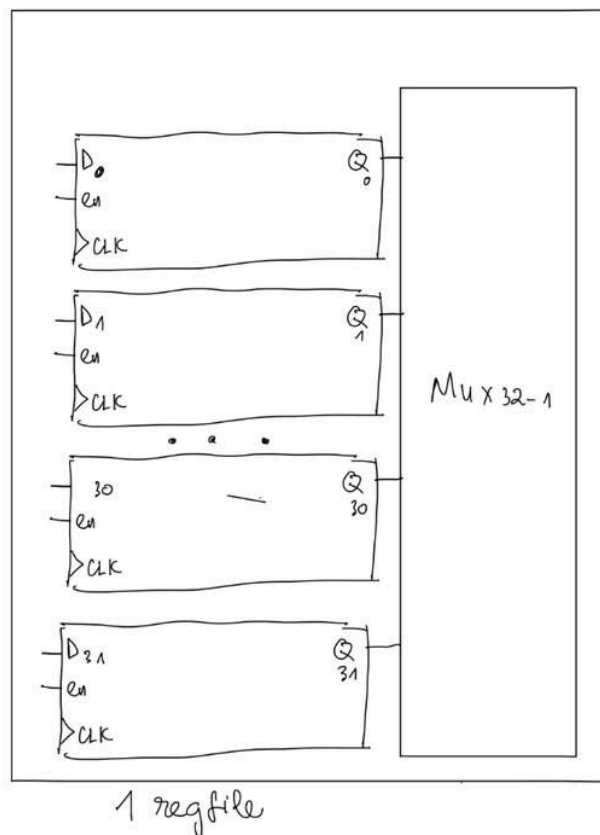
LESS

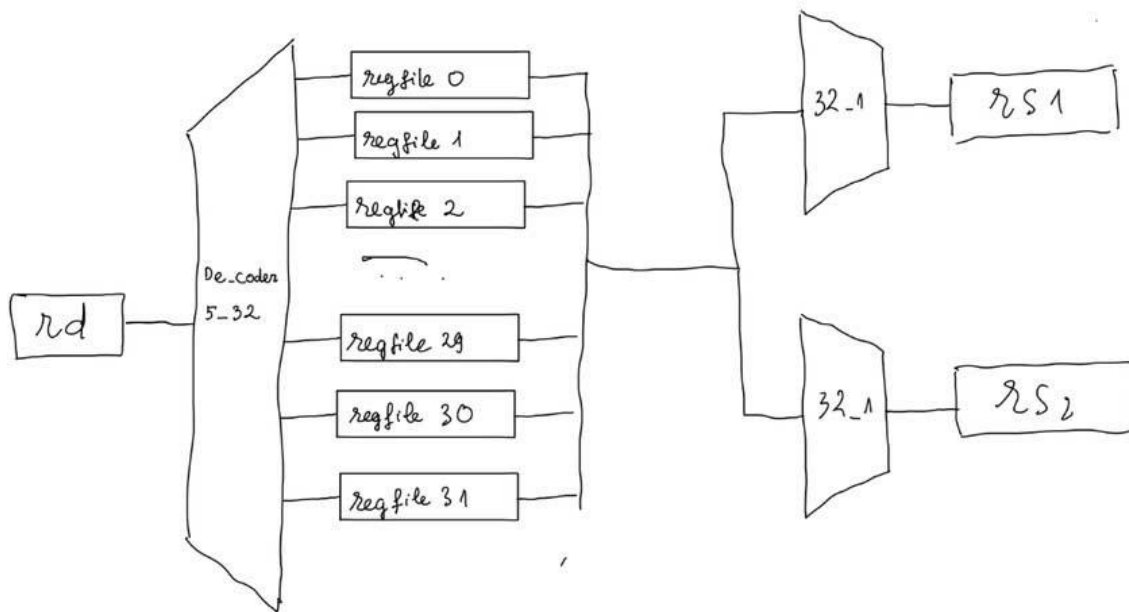


2.2. Regfile and Load-Store Unit.

❖ Phân tích yêu cầu thiết kế Regfile:

Yêu cầu thiết kế cho module "regfile.sv" theo chuẩn RISC-V bao gồm việc tạo một bộ thanh ghi với 32 thanh ghi 32-bit, trong đó thanh ghi 0 có giá trị là 0. Tên clock là "i_clk" và tên reset chế độ thấp là "i_reset", như đã được đề cập trước đó trong Milestone 1. Module này có các đầu vào như: tín hiệu clock positive, reset chế độ thấp, các địa chỉ cho việc đọc từ các thanh ghi (rs1_addr và rs2_addr), địa chỉ cho việc ghi vào thanh ghi (rd_addr), dữ liệu cần ghi (rd_data) và tín hiệu rd_wren để xác nhận việc ghi dữ liệu vào thanh ghi. Module cũng có các đầu ra là dữ liệu đọc từ rs1 (rs1_data) và rs2 (rs2_data). Bộ thanh ghi ghi dữ liệu vào tệp regfile.data. Dữ liệu ghi có sẵn để đọc ở chu kỳ tiếp theo mà không cần tín hiệu clock, trong khi quá trình đọc dữ liệu từ thanh ghi không cần tín hiệu clock.





❖ Phân tích thiết kế Instruction memory và Data memory:

Thiết kế một data memory có dung lượng 2KB, một instruction memory có dung lượng 8KB. Các Memory cần được thiết kế để có thể hiện thực được các lệnh lb(load byte), lh(load half word), lbu(load byte unsigned), lhu(load half word unsigned), lw(load word), sb(store byte), sh(store half word) và sw(store word). Để thực hiện được yêu cầu trên, memory được model dưới dạng các mảng 8-bit có một tín hiệu để lựa chọn đọc hoặc ghi data. Đối với data memory cần thêm 1 tín hiệu để lựa chọn kích thước data là byte, half word hoặc word và là số có dấu hoặc số không dấu.

❖ Phân tích thiết kế LSU

Thiết kế dựa trên mapping của đề kết hợp khối D-mem dùng funct3 để lựa chọn word, halfword hay byte bằng cách che bmask.

Dựa vào mapping :

Load từ địa chỉ ngoại vi = đọc ngoại vi

Store vào địa chỉ ngoại vi = ghi ngoại vi

→ LSU chỉ cần phân biệt:

Nếu addr thuộc vùng Data Memory → truy cập DMEM

Tạo decoder

0x1001_0000 → 0x1001_0FFF : Switches

0x1000_3000 → 0x1000_3FFF : 7-seg (7-4)

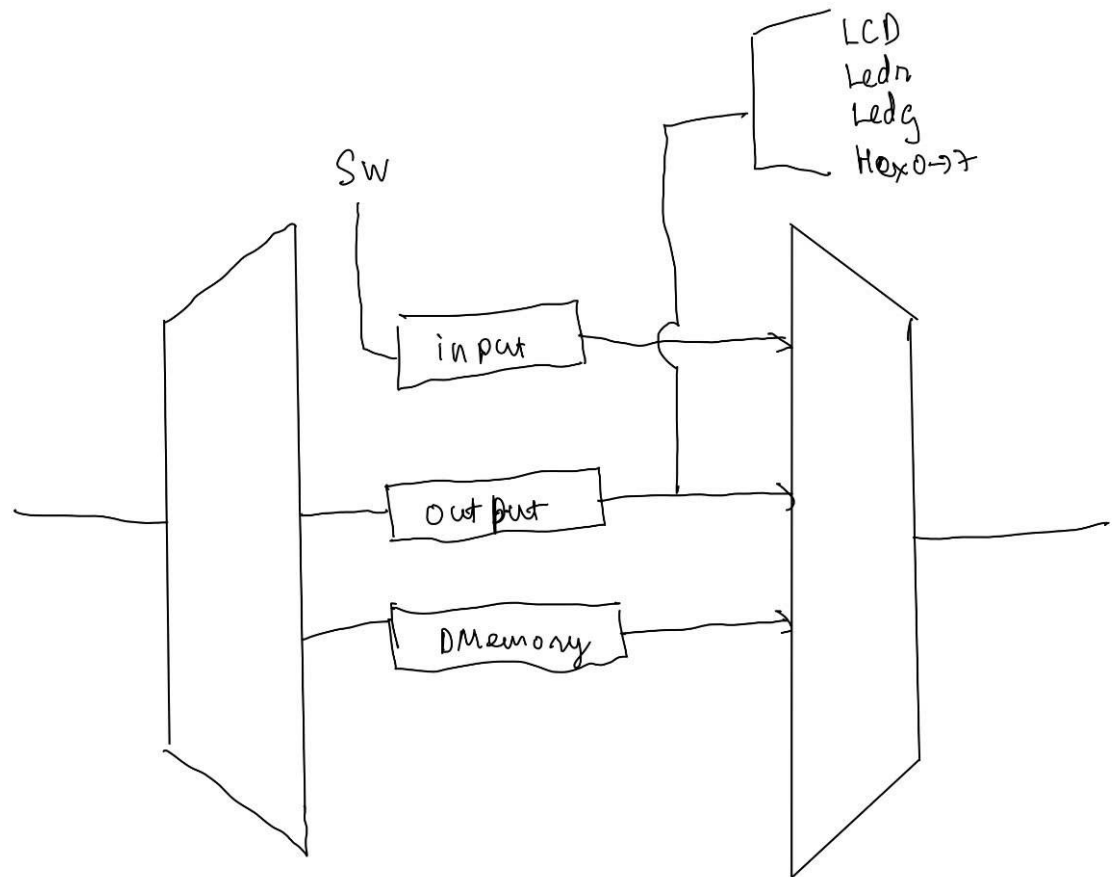
0x1000_2000 → 0x1000_2FFF : 7-seg (3-0)

0x1000_1000 → 0x1000_1FFF : Green LEDs

0x1000_0000 → 0x1000_0FFF : Red LEDs

0x0000_0000 → 0x0000_07FF : Data Memory 2KiB

0x1000_4000 → 0x1000_4FFF : LCD

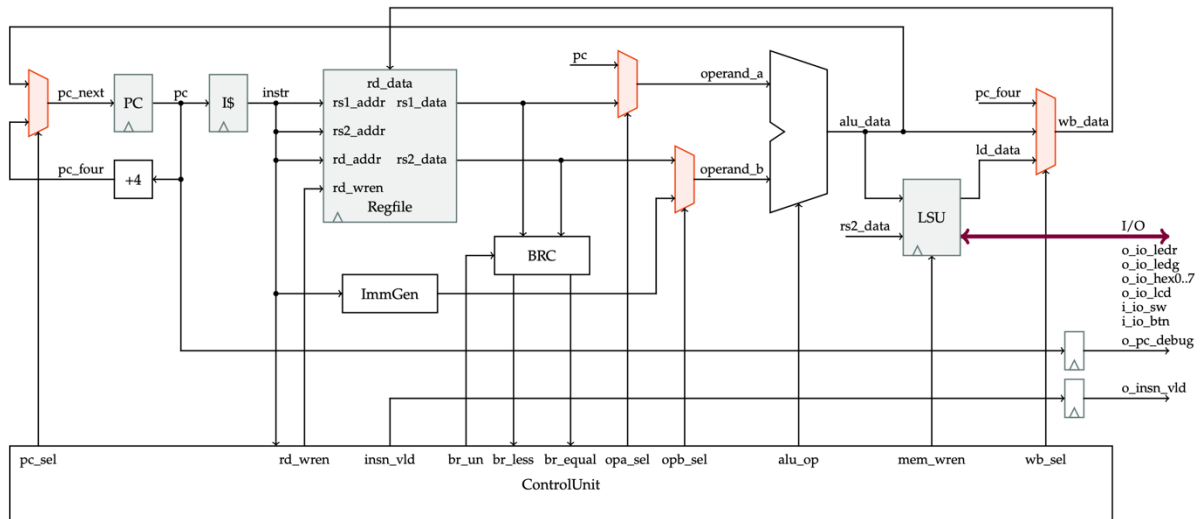


2.3. Single-Cycle Processor.

❖ Phân tích yêu cầu thiết kế Single-Cycle Processor:

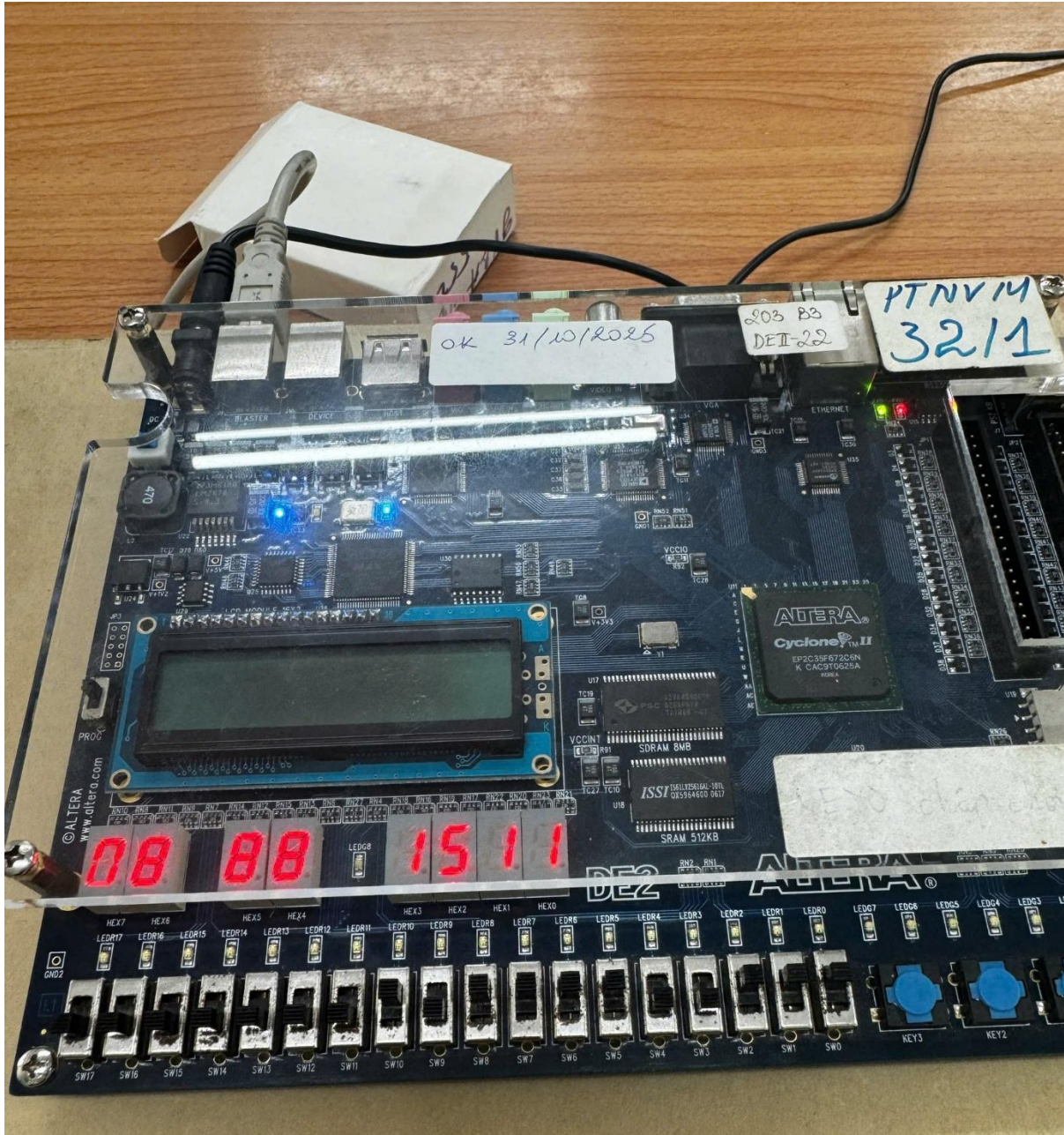
Từ những khối đã thiết kế trước đó, ta tiến hành ghép các khối lại thành một Processor

❖ Sơ đồ khối của Single-Cycle Processor:



3. Application

3.1 LED 7 đoạn : swith nhị phân ra led 7 đoạn thập phân



Video thực tế : [LINK](#)

Code assembly

main:

Ghi bảng mã led 7 đoạn cho các số 0–9 vào dmem (mỗi mã 4 byte)

addi x8, x0, 0x0C0

sw x8, 0(x0) # 0: mã số 0

addi x8, x0, 0x0F9

sw x8, 4(x0) # 4: mã số 1

addi x8, x0, 0x0A4

sw x8, 8(x0) # 8: mã số 2

addi x8, x0, 0x0B0

sw x8, 12(x0) # 12: mã số 3

addi x8, x0, 0x099

sw x8, 16(x0) # 16: mã số 4

addi x8, x0, 0x092

sw x8, 20(x0) # 20: mã số 5

addi x8, x0, 0x082

sw x8, 24(x0) # 24: mã số 6

addi x8, x0, 0x0F8

sw x8, 28(x0) # 28: mã số 7

addi x8, x0, 0x080

sw x8, 32(x0) # 32: mã số 8

```
addi x8, x0, 0x090
sw  x8, 36(x0)    # 36: mã số 9
```

Vòng lặp chính: đọc N, tách 4 chữ số, mã hóa ra led 7 đoạn và xuất
LOOP:

```
# Đọc giá trị N từ vùng vào (ví dụ: địa chỉ 0x10010000)
lui  x10, 0x10010    # x10 = 0x10010000
lw   x10, 0(x10)     # x10 = N (raw)
```

```
# Mask 17 bit thấp: N = N & 0x1FFFF
lui  x11, 0x20       # x11 = 0x00020000
addi x11, x11, -1    # x11 = 0x0001FFFF
and  x10, x10, x11    # x10 = N (17 bit thấp)
```

```
#-----
# 1. Chia cho 1000 -> lấy chữ số hàng nghìn
#-----
li   x12, 0          # x12 = digit_thousands
li   x16, 1000       # x16 = 1000
```

```
loop_1000:
    blt x10, x16, end_1000 # nếu N < 1000 thì dừng
    sub x10, x10, x16      # N -= 1000
    addi x12, x12, 1       # digit_thousands++
    j   loop_1000
```

```
end_1000:
    # x12 = hàng nghìn, x10 = phần dư (0..999)
```

===== CHECK OVERFLOW SAU KHI TÁCH HÀNG NGHÌN =====

Nếu hàng nghìn > 9 => số gốc >= 10000 => ép hiển thị 0000

li x16, 9

ble x12, x16, OK_4DIG # nếu x12 <= 9 thì là số <= 9999 → xử lý tiếp

Overflow: ép 4 digit = 0

li x12, 0 # hàng nghìn = 0

li x13, 0 # hàng trăm = 0

li x14, 0 # hàng chục = 0

li x15, 0 # hàng đơn vị = 0

j ENCODE_4DIG # bỏ qua chia 100/10, nhảy sang mã hóa led

OK_4DIG:

#-----

2. Chia phần dư cho 100 -> lấy chữ số hàng trăm

#-----

li x13, 0 # x13 = digit_hundreds

li x16, 100 # x16 = 100

loop_100:

blt x10, x16, end_100 # nếu N < 100 thì dừng

sub x10, x10, x16 # N -= 100

addi x13, x13, 1 # digit_hundreds++

j loop_100

end_100:

x13 = hàng trăm, x10 = phần dư (0..99)

#-----

3. Chia phần dư cho 10 -> lấy chữ số hàng chục

#-----

li x14, 0 # x14 = digit_tens

li x16, 10 # x16 = 10

loop_10:

blt x10, x16, end_10 # nếu N < 10 thì dừng

sub x10, x10, x16 # N -= 10

addi x14, x14, 1 # digit_tens++

j loop_10

end_10:

x14 = hàng chục, x10 = phần dư (0..9)

#-----

4. Hàng đơn vị = phần dư cuối

#-----

mv x15, x10 # x15 = digit_units

#-----

Tra bảng mã 7 đoạn trong dmem theo từng digit

dmem base = 0, mỗi phần tử là 1 word (4 byte)

địa chỉ = digit * 4 = digit << 2 = digit*2*2

#-----

ENCODE_4DIG:

addi x17, x0, 0xFF # mask 8 bit thấp

Hàng nghìn -> mã led ở x19

add x18, x12, x12 # x18 = digit_thousands * 2

```
add x18, x18, x18    # x18 = digit_thousands * 4 (byte offset)
lw  x19, 0(x18)
and x19, x19, x17
```

```
# Hàng trăm -> mã led ở x20
```

```
add x18, x13, x13
add x18, x18, x18
lw  x20, 0(x18)
and x20, x20, x17
```

```
# Hàng chục -> mã led ở x21
```

```
add x18, x14, x14
add x18, x18, x18
lw  x21, 0(x18)
and x21, x21, x17
```

```
# Hàng đơn vị -> mã led ở x22
```

```
add x18, x15, x15
add x18, x18, x18
lw  x22, 0(x18)
and x22, x22, x17
```

```
#-----
```

```
# Ghép 4 byte mã led thành 1 word:
```

```
# [x19|x20|x21|x22] -> x26
```

```
#-----
```

```
slli x23, x19, 24    # mã hàng nghìn << 24
slli x24, x20, 16    # mã hàng trăm << 16
slli x25, x21, 8     # mã hàng chục << 8
```



```
add x26, x23, x24
```

```
add x26, x26, x25
```

```
add x26, x26, x22    # x26 = word mã 4 led
```

```
#-----
```

```
# Ghi ra thanh ghi xuất led (ví dụ tại 0x10002000)
```

```
#-----
```

```
lui x27, 0x10002    # x27 = 0x10002000
```

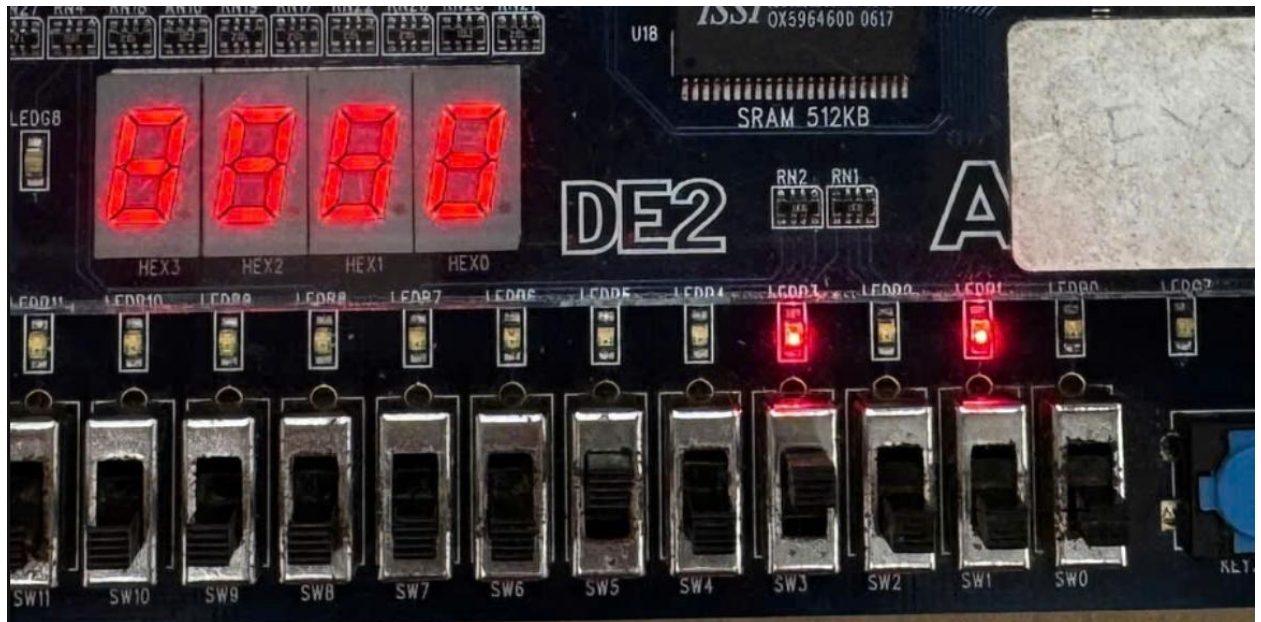
```
sw x26, 0(x27)    # ghi mã 4 led
```

```
# Lặp lại liên tục
```

```
jal x0, LOOP
```

3.2 Led red : tính toán 4 bit + -

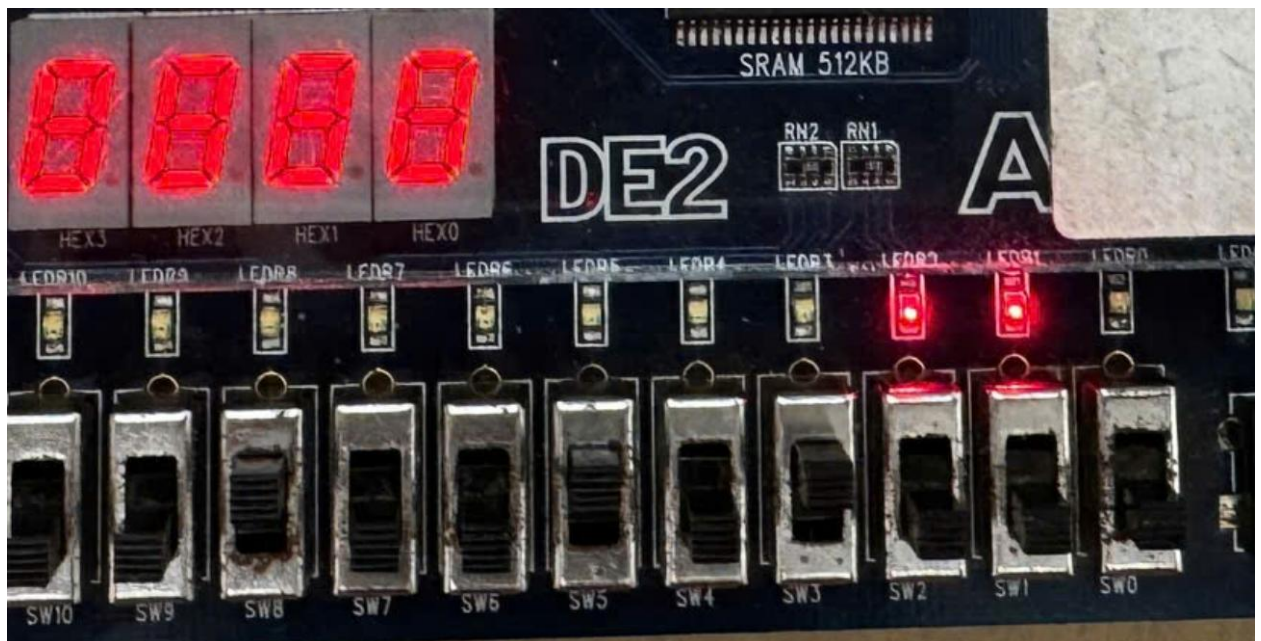
Gán 4 sw đầu là A , 4 sw tiếp theo là B , sw 8 là select + hay -



- Gạt sw3 (A = 8) + gạt sw5 (B= 2) và sw8 không gạt

→ Bộ cộng

→ $8+2=10$ → nhị phân là 1010



- Gạt sw3 (A = 8) + gạt sw5 (B= 2) và sw8 có gạt lên

→ Bộ trừ

→ $8-2=6$ → nhị phân là 0110

Assembly

0x0	0x100102B7	lui x5 65552	lui x5, 0x10010 # base address of SW (0x10010000)
0x4	0x10000337	lui x6 65536	lui x6, 0x10000 # base address of LEDR (0x10000000)
0x8	0x0002A383	lw x7 0(x5)	lw x7, 0(x5) # đọc giá trị tất cả SW
0xc	0x00F3F413	andi x8 x7 15	andi x8, x7, 0xF # SW0–SW3 => A
0x10	0x0043D493	srli x9 x7 4	srli x9, x7, 4
0x14	0x00F4F493	andi x9 x9 15	andi x9, x9, 0xF # SW4–SW7 => B
0x18	0x0083D513	srli x10 x7 8	srli x10, x7, 8
0x1c	0x00157513	andi x10 x10 1	andi x10, x10, 1 # SW8 => bit điều khiển (0 cộng, 1 trừ)
0x20	0x00050863	beq x10 x0 16	beq x10, x0, cong # nếu x10 == 0 → cộng
0x24	0x409405B3	sub x11 x8 x9	sub x11, x8, x9 # nếu x10 == 1 → trừ
0x28	0x00B32023	sw x11 0(x6)	sw x11, 0(x6)
0x2c	0xFDFF06F	jal x0 -36	jal x0, loop
0x30	0x009405B3	add x11 x8 x9	add x11, x8, x9
0x34	0x00B32023	sw x11 0(x6)	sw x11, 0(x6)
0x38	0xFD1FF06F	jal x0 -48	jal x0, loop